

How Hardware Gets Hacked (Part 8)

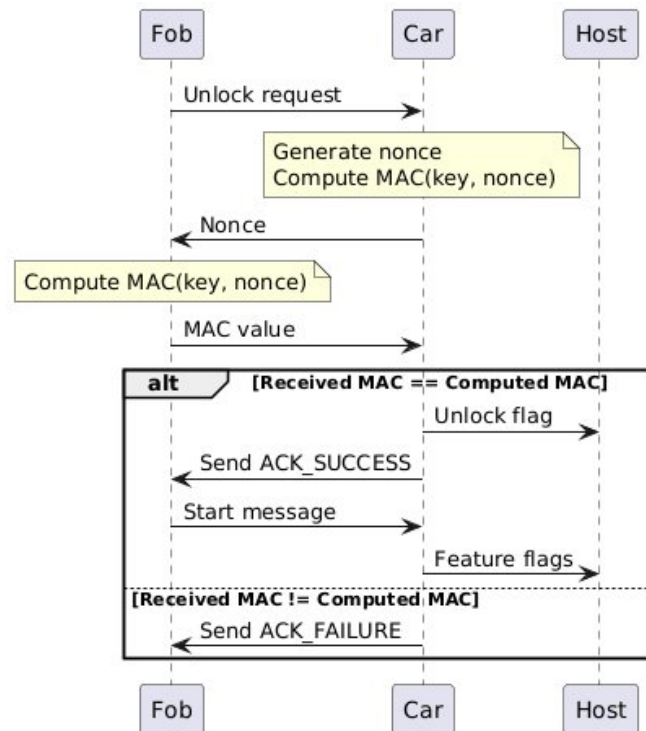
Brute Force and Timing Attacks

by Nathan Jones

Introduction	1
Another glaring vulnerability	2
Attack #4: Brute force the pairing pin.....	4
Protecting against brute force attacks	5
Defense #4: Timeouts.....	7
Side-channel leakage	8
Are all of our byte-by-byte comparisons vulnerable to timing attacks?	12
Attack #5: Timing attack on the pairing pin.....	13
Defense #5: Compare pins in constant-time.....	14
Pitfalls	16
Updated threat model.....	20
Full threat model.....	21
Conclusion.....	21

Introduction

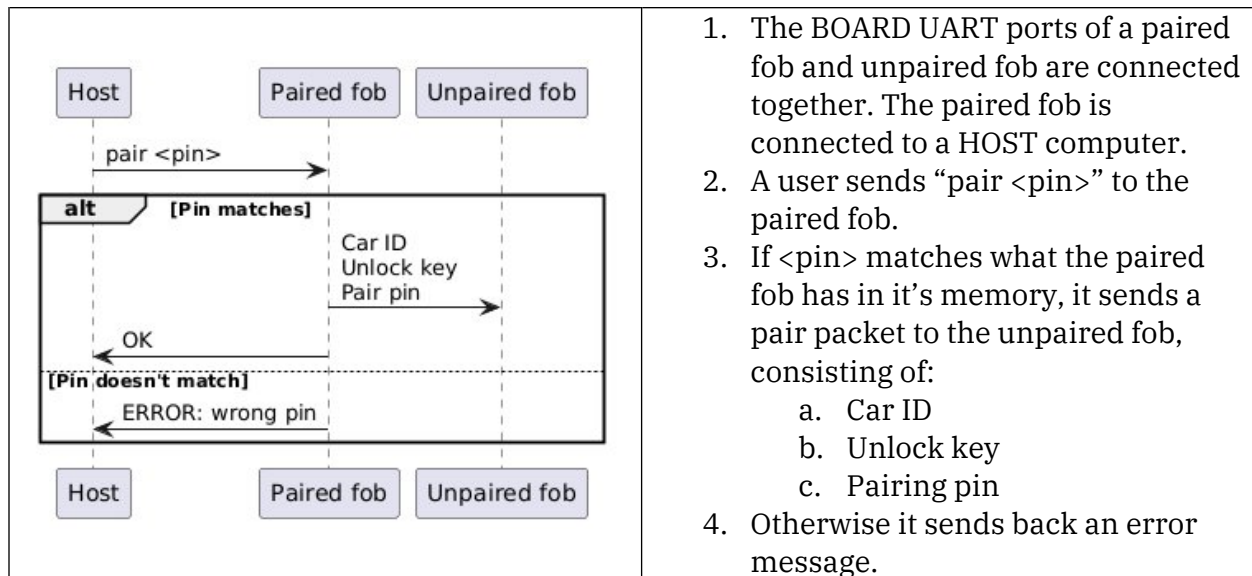
At this point our unlocking process is fairly secure. In the [last three articles](#), we built a system in which a car and paired fob each get a secret key stored in flash during build-time. The car, at start-up, gathers entropy from various sources and uses that to seed it's random-number generator. When a fob wants to unlock it, the car sends the fob a random number (a.k.a. “nonce”), over which the fob computes the MAC (message authentication code) value using the secret key. The fob sends the MAC value back to the car and if the car gets the same value when it computes it's own MAC value, it unlocks.



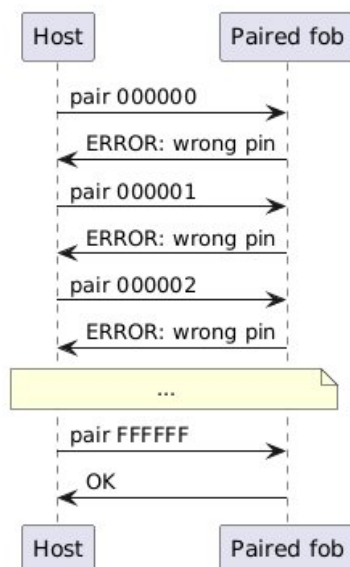
At this point an attacker would need to find a way to do something crazy, like break AES-CMAC to forge a MAC value or make a nonce repeat more than once, in order to defeat our authentication attempts. Both of those seem pretty difficult, so let's move to a different part of the codebase that presents a much easier target: **the pairing process**. We'll soon discover that this feature is about as insecure as the unlocking process originally was, easily allowing an attacker to extract the pairing pin or the unlock key in a matter of minutes to hours using either brute force or timing attacks. (You can bet that a real attacker is going to zero in on *exactly* the weakest part of your system that gives them the payoff they want!) Along the way, we'll learn about side-channel attacks and constant-time comparisons.

Another glaring vulnerability

Let's review the general pairing process.



Per the competition rules, the pair pin is required to be “6 hex digits”, which gives us about 16.7 million possible pin combinations ($16^6 = 16,777,216$). This might seem like a lot of combinations, but it’s really not, at least not if we are thinking in terms of computer speeds. Sure, if you and I were making blind guesses at the pin we might only be able to make one attempt every second, meaning it could take us as long as 194 straight days (without food, sleep, or bathroom breaks) to find the correct pin. But if an attacker were to write a script that could make each guess in **2.5 ms** (accounting for just the round-trip time to send the pair message and then receive the ensuring error message, ERROR: wrong pin), it might only take them **11.7 hrs** to guess the correct pin.



Note: These are worst-case time estimates ($1 \text{ sec} * 16^6 \text{ possible pins} = 194.18 \text{ days}$; $2.5 \text{ ms} * 16^6 = 11.65 \text{ hours}$) for pin values that are equal to the last value checked by an attacker (as depicted in the sequence diagram above). In reality, an attacker has a 50% chance of guessing a random pin in only half the time, so more like 5.9 hours to brute force our pairing pin (on average).

At that point the attacker would know not only the pairing pin but also the unlock key, allowing them to unlock that car at will by computing the MAC value of any nonce challenge (and also pair any unpaired fob they wanted to that car). This is called a **brute force** attack, since it involves no complexity, just guessing pin combinations until one of them succeeds.

Attack #4: Brute force the pairing pin

As a working paired fob is needed to conduct this brute force attack, we are targeting **Cars #2 and 5** from the competition.

Scripting the attack is exceedingly straightforward. The pseudocode might look like this:

```
FOREACH pin_value
    SEND "pair <pin_value>"
    IF resp == "OK":
        // We win!
```

As mentioned above, however, this test could take up to 11.7 hours to run, which is far too long for an automated test. (Though possibly we could add this to a suite of tests that ran only periodically, such as once per week.) Instead, we'll set the pin in our test to some low value and check that a brute force attack can succeed within a certain small amount of time. If, for instance, the pin is set to 0x00000A then our design should most certainly fail if the test can brute force that pin within 25 ms (i.e. 2.5 ms per guess, extrapolating to a worst-case brute force time of 11.7 hours). If it takes some amount longer, then the design should pass (we'll discuss how long is "long enough" in the next section).

As expected, our current firmware fails this test (each pairing attempt only took $495 \mu\text{s}$ in simulation!).

```

> pytest testing/security_tests.py::TestPairingPinAttacks::test_brute_force_attack_on_pairing_pin
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/security_tests.py F

===== FAILURES =====
testing/security_tests.py:270: in test_brute_force_attack_on_pairing_pin
    assert (end - start) > 5, f"Pairing attempts not sufficiently slow ({(end - start)/10} seconds per attempt)."
E   AssertionError: Pairing attempts not sufficiently slow (0.000494462699862197 seconds per attempt).

```

/*****[? Does this mean we could brute force an unlock attempt?]***/

No. I mean, we could try, but it wouldn't be nearly as effective. There are two possible targets for a brute force attack during the unlocking process:

1. After observing a fob compute a single MAC value, test all possible unlock keys to find the one that produces the same MAC value.
2. After a car sends out a nonce, try every possible value until it finally accepts one.

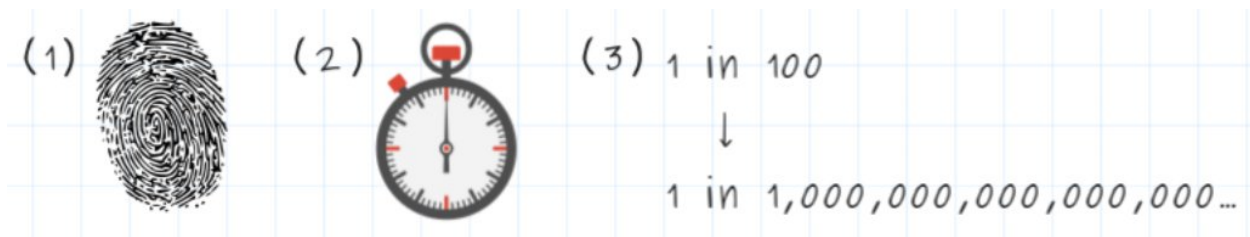
Neither one is realistically feasible. In the first case, an attacker would potentially have to test all 2^{128} possible unlock keys to perform this attack. Even if they could test *one million* keys a second, it would still take them over ***10²³ centuries*** to find the correct value. And in the second case, the attacker only actually gets one attempt per nonce. Not only would they be trying to guess the correct one of 2^{64} possible MAC values for a given nonce, but they would have to trigger enough unlocks for the car's PRNG to use the same nonce a second time before they ever got another attempt at a guess. NGL, I don't even want to know how many zeros are on the "length of time needed to brute force" that value.

*****/

Protecting against brute force attacks

Brute force attacks work because the attacker can make so many attempts, so quickly, that it overcomes the odds (in spite of the odds of each individual attack being so abysmally low). It would be like playing the lottery, except instead of buying a single ticket or even twenty, you bought 1,000,000,000; the odds are the same, but your overall chances of success would go *way* up.

To defend against a brute force attack, we have to push the odds back in our favor. Possible defenses fall into one of three camps.



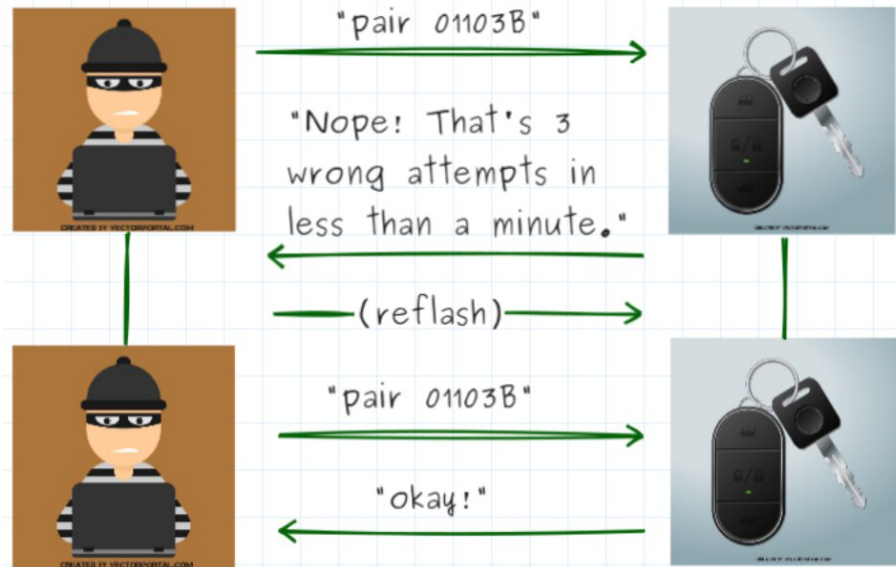
1. **Make each attempt categorically more difficult:** E.g. require an attacker to provide a fingerprint scan alongside the pin or lock/erase the device after 8 unsuccessful attempts within a minute. These elevate the complexity of the attack, either on every attempt or after X attempts (that trigger a lockout/erase).
2. **Make each individual attempt take longer:** Enforce a rate-limit of one attack per second or have a 1 minute timeout after 3 unsuccessful pairing attempts. These greatly increase the amount of time a successful attack could take.
3. **Decrease the odds of any one guess being successful:** Increase the pin from 6 hex digits to 16; the number of possible combinations would go up to 1.84×10^{19} and even if each attempt only took 2.5 ms it could still take an attacker up to *1.46 billion years* to find a correct guess. (This is the same reason why it's unfeasible to conduct a brute force attack on either the unlock key or the MAC value, as discussed above ["Could we brute force the unlocking process?"].)

Unfortunately, we can't change the competition rules so options 1 and 3 are non-starters (but great ideas to keep in mind for other production devices!). We'll pursue option 2 in this article.

/*****[! Log, report, and react to suspicious behavior]*****/
 Just like your device (hopefully) reacts to exceptional or erroneous behavior (like inputs being out of range), it's also good practice to look for and react to suspicious behavior like repeated incorrect pairing attempts or pairing attempts that occur within a small window (which could be indicative of a scripted attack). Even if you don't necessarily do anything proactive with that information like enforce a timeout or erase the device (as mentioned above), you can at least log it like any other error (even by just incrementing a counter) or report it back to a central server.

*****/

One complication, though, is that the attacker has control of our hardware, meaning they can reset and/or re-flash our fob at-will. If our "number of unsuccessful pair attempts" variable lives in RAM, a simple reset will clear it out. In fact, even if we store this variable in flash to survive a power cycle, a re-flashing (or a mass erase and then re-flashing) will *still* overwrite that variable!



The best way to get around this is to not store a variable at all: simply enforce a 1 second timeout **every time the user tries to pair** (or, rather, a “1 second minus time-to-pair” timeout). This delay adheres to the rules and would be meaningless to an actual user, while forcing each pairing attempt to take at least that long to complete. At 1 second per attempt (up from 2.5 ms), the maximum amount of time needed to conduct a brute force attack goes from 11.7 hours to **194 days**.

/*****[? How long is long enough?]***/

Depending on the production device, 194 days to brute force a pin may not be long enough. An attacker who gets a job working at a car rental company or managing a company’s fleet of vehicles, for instance, could easily have access to a fob for 194 days or longer. If that’s the case, then what length of time do you think is long enough to be “infeasible” for an attacker to brute force? How big would the pin need to be in order for brute forcing to take that long (assuming an attacker can check one pin guess every 2.5 ms, as above)? (Possible answerⁱ)

*****/

Defense #4: Timeouts

We’ll make our delay 0.75 sec (to ensure we meet the 1 sec pairing deadline), added as a blocking delay at the top of pairFob.

```
void pairFob(FOB_FLASH_DATA *fob_state_ram, const char *pin)
{
    delay_ms(750);

    // Only paired fobs can initiate pairing
    if (fob_state_ram->paired != FLASH_PAIRED)
    {
        sendError("not paired");
    }
}
```

Adding that delay to each pairing attempt (and providing the proper implementations for it on each of our hardware platforms, of course) allows us to pass our test!

```
> pytest testing/security_tests.py::TestPairingPinAttacks::test_brute_force_attack_on_pairing_pin
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/security_tests.py .

===== 1 passed in 15.34s =====
```

Side-channel leakage

Unfortunately, we're not out of the woods yet. Right now, we're assuming that the attacker gets no information from a wrong guess other than that it's wrong (ERROR: wrong pin). So wouldn't it be awful if our system was actually playing a game of Wordle (or Mastermind, for you geezers) with the attacker while they were making guesses?

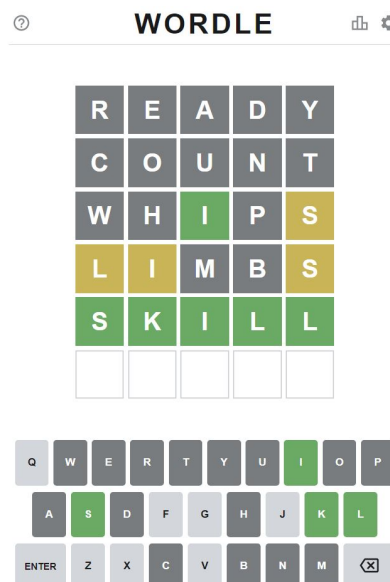


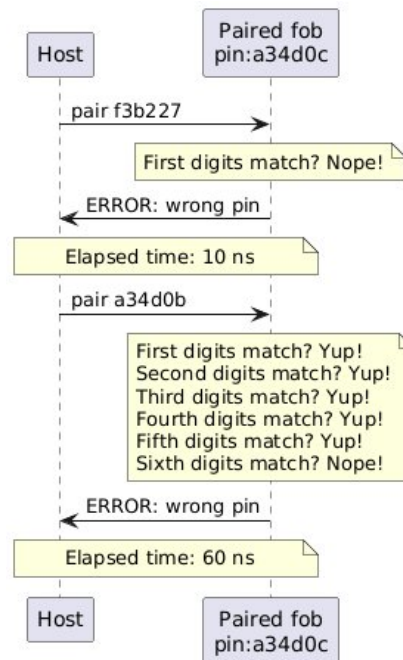
Figure 1: https://upload.wikimedia.org/wikipedia/commons/9/9d/Wordle_6._veebbruar_2022.jpg

“Ooh, ‘1a2b3c’ is so close! There’s *definitely* a 1, A, and an C in the actual pin, just not in those exact positions.”

That would give the attacker a *significant* advantage in determining the correct key. They wouldn’t need to guess randomly anymore, but could guess *selectively* based on the clues given to them. This would greatly reduce the total number of guesses they would need to make to get to the correct pairing pin.

And, yes, our system doesn’t do this *exactly*. But it *does* give our attackers more information than we’d intended in the form of **how long it takes to get an error response**.

Think about how the fob is checking a pin’s correctness: in all likelihood, it’s starting at the first byte and seeing if it matches what it has in memory, going on to the next character if it is or sending an error message as soon as it isn’t. This is just good programming! Why check any remaining digits if you already know that the pins don’t match?



The problem is that there’s now a difference in how long it takes to get an error message from the fob, depending on how many digits in the wrong pin were actually correct.

Do none of the digits match? The error message comes out really fast. Do all the digits match except the last one? Then that error message comes out later than the first by the amount of time it took the fob to do 5 byte comparisons. The amount of time might not be long, tens of nanoseconds possibly, but it’s consistent and absolutely measurable.

The attack works like this:

1. Make 16 pin guesses, each different in their first digit (the value of the remaining digits doesn't matter at this point)
2. Measure how long it takes each error message to come back. One of the 16 error messages will take slightly longer to come back as compared to the others; this corresponds to the first correct digit of the pin, since the fob identified that it matched the first digit in its memory and moved on to check the second digit before sending the error message.
3. Repeat for the remaining 5 digits.

With this attack an attacker only needs to make **96 guesses** to find the correct pin (16×6), as opposed to the 16.7 million guesses they needed to make in the beginning. (In fact, they only really need to make 48 guesses, on average. If they're checking each message's return time as it's being guessed then it will be fairly obvious that one guess, 0x600000 say, took longer to return than each of the guesses 0x000000 through 0x500000, making 6 the logical first digit.) Even if it took a *full minute* to make each guess, an attacker could use this to brute force the pin value in little more than *an hour and a half*.

This type of attack is called a **"timing attack"**, as it's related to the timing of certain critical MCU operations, and it's part of a broader class of attacks called **"side-channel attacks"**. A "side channel" is a way that an MCU communicates (or "leaks") information about what it's doing simply by virtue of doing it, like finding fingerprints at a crime scene.

- Is the MCU iterating more times than normal in a loop? → The output signal will be delayed by that amount of time (like we saw above).
- Is the MCU perform an intensive operation? → We should see an increase in power draw and heat generated during that time.

A side-channel attack simply turns this information into an actionable security vulnerability.

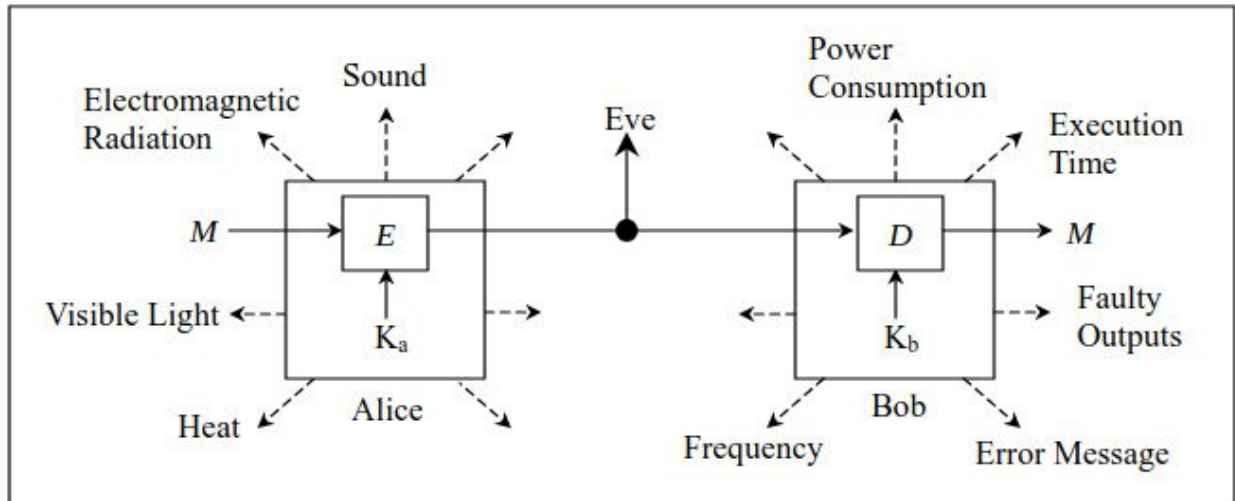
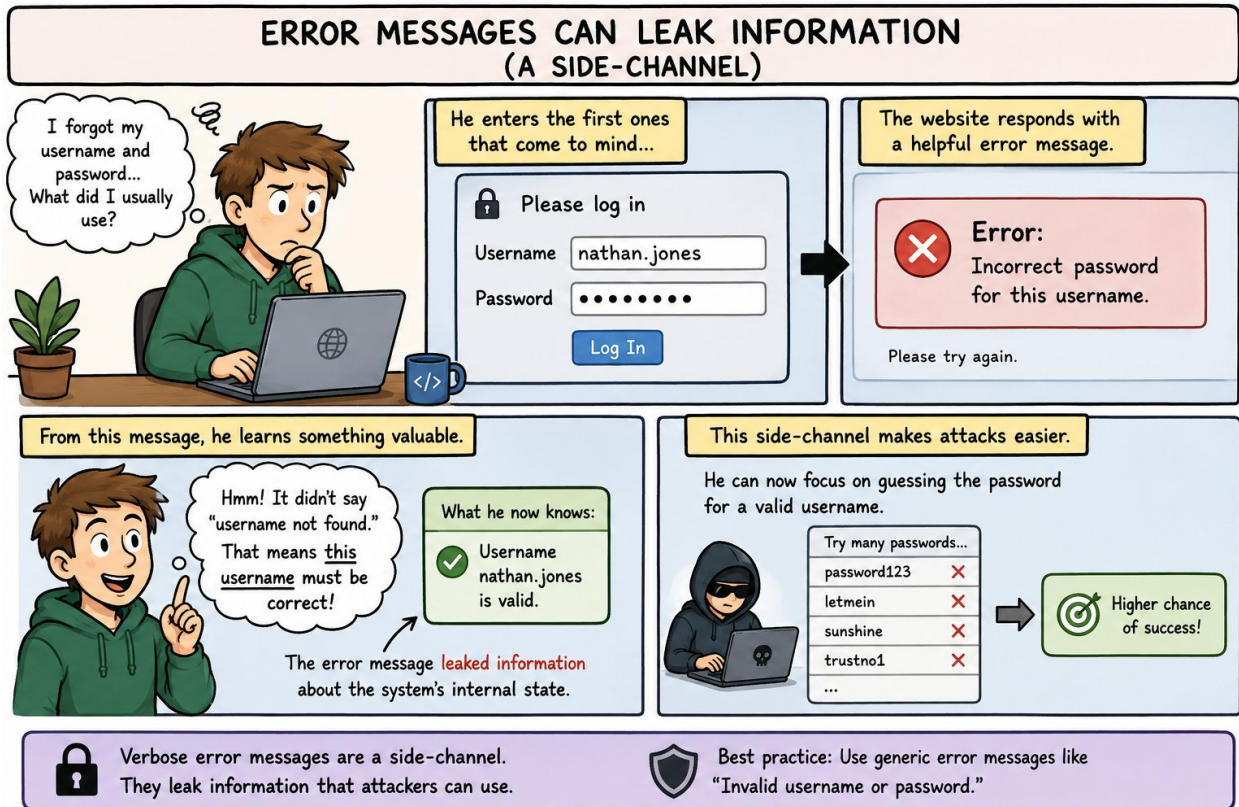


Figure 2: The cryptographic model including side-channel

Figure 2: Zhou, Yongbin & Feng, DengGuo. (2005). Side-Channel Attacks: Ten Years After Its Publication and the Impacts on Cryptographic Module Security Testing.. IACR Cryptology ePrint Archive. 2005. 388. <https://eprint.iacr.org/2005/388.pdf>

It turns out that there are several possible side channels on an MCU (see above), from timing to power to even the nature of any error messages sent back.

In fact, you may have used the nature of an error message to conduct a side-channel attack of your own! Have you ever tried to log in to a website, only to realize you'd completely forgotten your username and password, and decided to try the first ones that came to your mind? If you were greeted with "Error: Incorrect password for this username" and you thought "Aha! At least I guessed my username correctly" then you've conducted a successful side-channel attack on that website. The nature of the error message leaked information that your username was likely correct, and this may have helped you figure out how to log in.



Are all of our byte-by-byte comparisons vulnerable to timing attacks?

`memcmp` is used one other place in our code (by the car, when it compares the received MAC to its calculated MAC during the unlocking process) and `strcmp` in two places (once by the fob and once by the car, both to check if a car ID matches the one in their memory).

In the case of `memcmp`, I think these specific uses *aren't* vulnerable to a timing attack since the MAC value being compared to changes with each new nonce.

In the cases of the two `strcmp`s, I think these *could* be considered vulnerable to timing attacks. The only thing this would reveal, though, would be the car ID being used, which is already known to an attacker for each scenario.

```
/*****[✓ Add memcmp/strcmp to your recon checklist]*****/
```

In [Part 3 of this series](#), we talked about analyzing source code to look for “known bugs” and “interesting oddities”, things that could possibly turn into vulnerabilities. Although it may not have been on your list then, I hope now you’ll know to flag the use of `memcmp` or `strcmp` as an “interesting oddity”, since it could be possible to use a timing attack to

extract the value that's being compared to!

```
*****/
```

Another way signal timing could leak sensitive information is when an attacker-controlled value is compared to a secret, with the MCU taking either a very short or very long computation path depending on the comparison. For example:

```
#define secret_val 0xCA
if(input == secret_val) longOp();
else shortOp();
```

If we can tell when the MCU performs `longOp()`, we can send 256 different inputs to the MCU to find the one that corresponds to the long operation and we'd have found our secret value. Fortunately, I don't see any code like this in our current firmware.

So, no, we don't necessarily have anything more to worry about in our code, at least with regard to a timing attack. Side-channel attacks are a deep topic, however, and this won't be the last time they're discussed in this series.

```
/*****[✓ Check my work]*****/
```

Take a look at the firmware (in particular, [fob.c](#) and [car.c](#)) and convince yourself that we have no more timing-related leakage (or prove me wrong!).

```
*****/
```

Attack #5: Timing attack on the pairing pin

To conduct our timing attack in a real scenario, we'd probably bust out our logic analyzer and write a script to get accurate timing of the error message, as measured from the STOP bit of the last byte in our UART pairing message until the start of the ensuing response from the fob. The pseudocode to conduct our timing attack might look like this:

```
guess = ""
FOREACH digit 1..6
    msg_times = [0]*16
    FOREACH hex_value 0..F
        Set up capture
        SEND "pair <guess + hex_value + 0*(5-len(guess))>"
        msg_times[hex_value] = message time
    Append hex value corresponding to longest message time to "guess"
```

I'd like to be able to run these security tests without needing extra hardware, however, so I'm going to take a few liberties in how this test is conducted. For the test I'm going to write, I'm going to slightly modify the pairFob function so that it calculates the execution time of memcmp and stores it in a local variable (provided TEST_BUILD has been defined).

```
#ifdef TEST_BUILD
    uint32_t start = getHardwareTime();
#endif

    bool pins_match = (memcmp_ct(hex_pin, fob_state_ram->pair_info.pin, 3) == 0);

#ifdef TEST_BUILD
    last_memcmp_execution_time = getHardwareTime() - start;
#endif
```

A new test command, getMemcmpTime, returns this value over the HOST UART, which is how our tests will “measure” each message time.

As expected, our current firmware fails this test.

```
> pytest testing/security_tests.py::TestPairingPinAttacks::test_timing_attack_on_pairing_pin --using tm4c@0E236CE6,0E239FA4
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/security_tests.py F

===== FAILURES =====
testing/security_tests.py:305: in test_timing_attack_on_pairing_pin
    assert guess != random_pin.upper(), "Pairing pin was recoverable using a timing attack"
E   AssertionError: Pairing pin was recoverable using a timing attack
E   assert 'DAD134' != 'DAD134'
```

Note that this test only works on hardware that has timing circuitry with a resolution of a single clock cycle: the STM32 or the TM4C. Unfortunately, there is no way to get cycle-accurate values in simulation; even clock_gettime only has a resolution of a few dozen nanoseconds, which isn't going to resolve the effect of the extra 2-3 machine instructions of the first digit matching. So this test is skipped if there is no hardware present.

Defense #5: Compare pins in constant-time

The best defense against a timing attack like this is to perform the byte-by-byte comparison of the pins in **constant-time**, removing any early termination entirely. A constant-time version of memcmp might look like this:

```
int memcmp_ct(const void *ptr1, const void *ptr2, size_t num)
{
    int diff = 0;
```

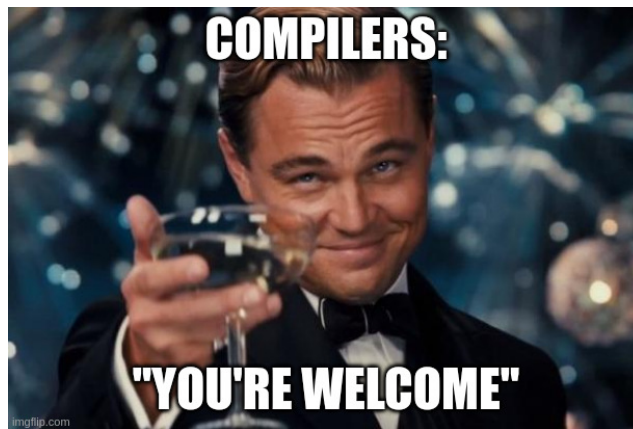
```

for(size_t idx = 0; idx < num; idx++)
{
    uint8_t byte1 = ((const uint8_t*)ptr1)[idx];
    uint8_t byte2 = ((const uint8_t*)ptr2)[idx];
    diff |= (byte1 ^ byte2);
}
return diff;
}

```

If the respective bytes of `ptr1` and `ptr2` differ at all then the result of XORing them together will have at least one 1 in it, so `diff` will be non-zero at the end of the loop. If `diff` equals zero, it means that every byte of `ptr1` exactly matched `ptr2`. Furthermore, this comparison takes the same amount of time, regardless of how `ptr1` and `ptr2` differ, which eliminates any timing leakage from our pin comparison operation. Another option is to use a pre-written function, such as [ConstantCompare\(\) from wolfSSL](#), instead of writing our own.

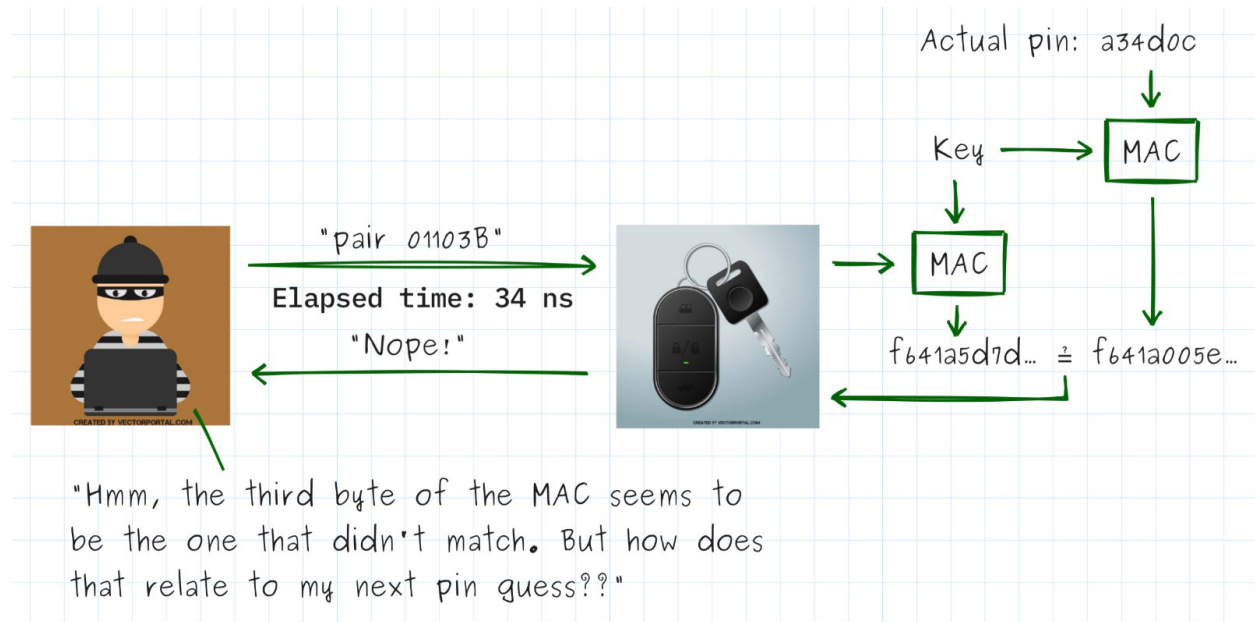
One pitfall to watch out for are **compiler optimizations**, which may emit assembly code that *won't* actually run in constant-time, foiling our defense! The compiler is free to do this because it's only goal is to convert our source code into assembly code that is fast, small, and, most importantly, *correct*.



There's simply no concept of code having "constant-time execution" in C/C++, no way to express as such in either language (or even to use any compiler extensions to do so). The fact that the code above *does* seem to have this property is an unenforceable side-effect of the way it was written, and should be verified by looking at the actual assembly code that gets generated.

An alternative to doing a constant-time comparison is to **compute the MAC value** of the received pin and **compare that to the MAC value of the stored pin**. The comparison will

still leak its value via a timing attack, but the information leaked will be of no use to an attacker.



Any cryptographically-secure MAC function will produce outputs that look seemingly random, even for inputs that only differ by a single bit (on top of which, our attacker can't see the outputs of the MAC functions, anyways); this is called the “avalanche effect”, which was mentioned in [Part 7 of this series](#). This means that even if the MAC comparison happens byte-by-byte (i.e. not in constant-time), the attacker has no idea *which* byte could have caused a failing comparison, only that a pin comparison has failed, and they're reduced to trying to brute force the entire pin space again. Note that this would need its own key, separate from the unlock key, and it might still be a good idea to do the MAC comparison in constant-time, anyways.

I'll stick with `memcmp_ct` for the simplicity. After replacing our library version of `memcmp` with this constant-time version, the test now passes.

```
> pytest testing/security_tests.py::TestPairingPinAttacks::test_timing_attack_on_pairing_pin --using tm4c00E236CE6,0E239FA4
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/security_tests.py .

===== 1 passed in 8.24s =====
```

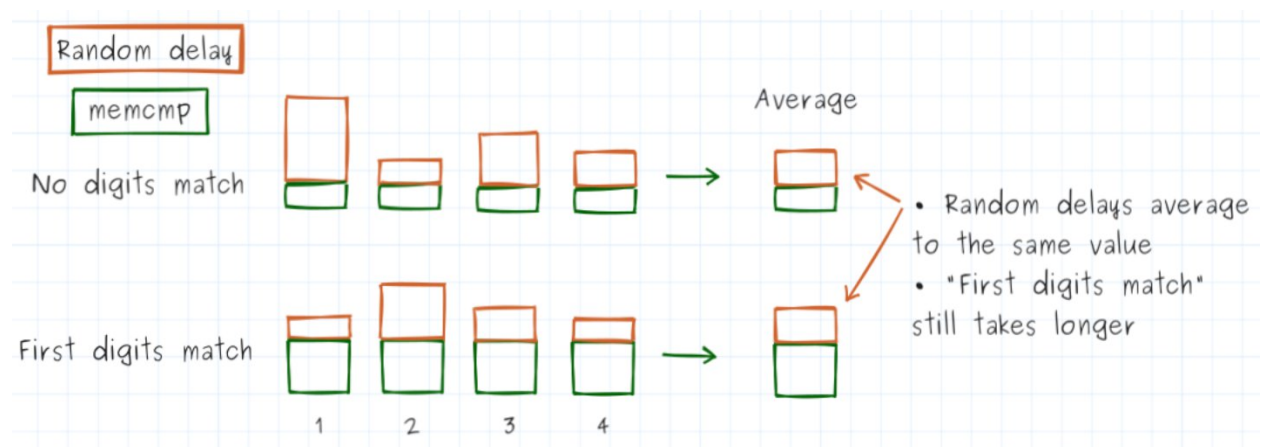
Pitfalls

There are a handful of pitfalls to avoid when implementing this defense, or another aimed at defeating a timing attack.

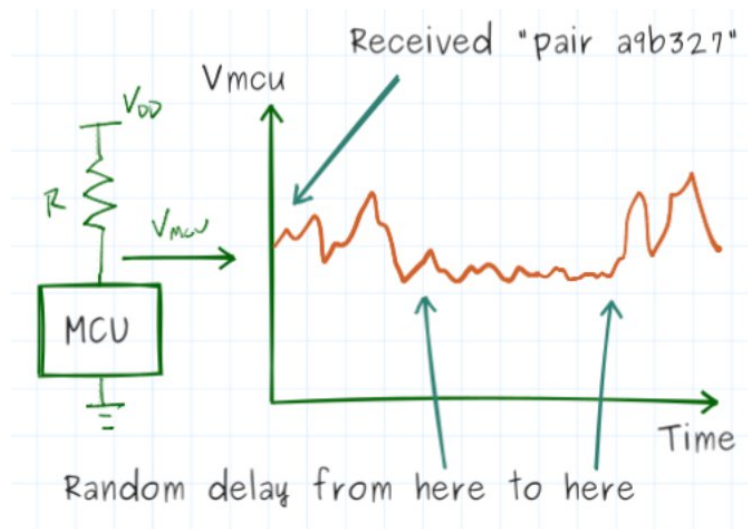
One such pitfall is **adding a random delay** somewhere in `pairFob()`.

```
void pairFob(FOB_FLASH_DATA *fob_state_ram, const char *pin)
{
    delay_ms(rand());
    ...
}
```

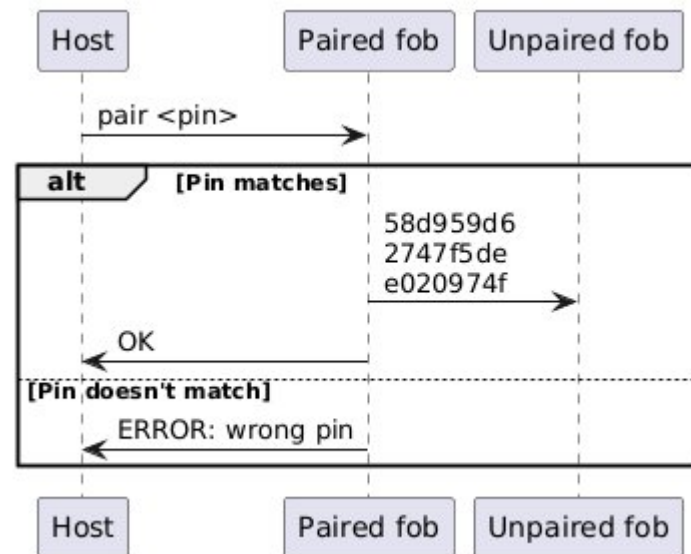
Although this would thwart our current attack, the fact of the matter is that all we would have done is added a noisy signal on top of our still-very-consistent timing information. And all we would need to do to extract that timing information is take the average of many, many pin guesses to average out the noise being added by the random delay.



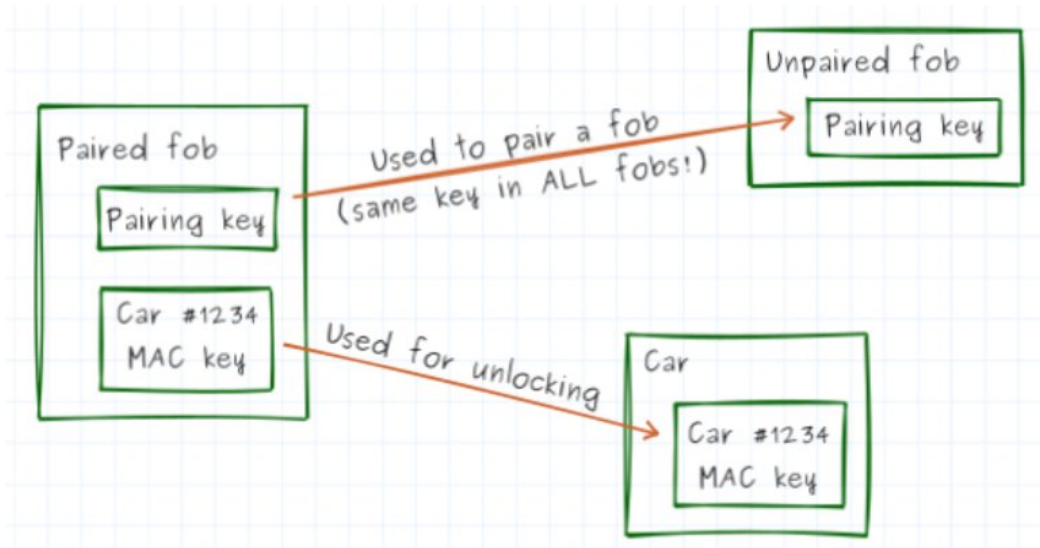
Furthermore, we might also be able to watch the MCU's power trace during a `pairFob` operation. During the `delay_ms` function the power draw would likely be fairly low and consistent (since the MCU isn't doing much during that time except spinning inside a blocking loop), which we could potentially see in that power trace. We could then simply start our timer from when the power trace changes, getting around the random delay. This is an example of a **power side-channel attack**.



Another pitfall is **encrypting the pair info** when it gets sent from a paired fob to an unpaired fob. This one seems so attractive! If we're worried about an attacker observing the pair info after forcing an unauthorized pair attempt, why not just hide the data from them completely?



In production, this may very well be a good strategy. However, one downside with this strategy is that **every fob** would need to carry the same secret key to do the encrypting and decrypting of the pair info.



Even on production devices, this sets up a “blast radius” problem: if *any* attacker is able to extract that secret key from *any* device, they can decrypt *every* pairing message for as far into the past and the future as that key has or will be used. Even if we believe this to be an unlikely scenario, it’s setting the system up for catastrophic failure if we’re ever proven wrong.

But we have an even bigger problem on the competition side: every attacking team is given an unencrypted copy of the binary for Car #0 and it’s paired fob.

Car 0: Your Car

The attacking team is provided with:

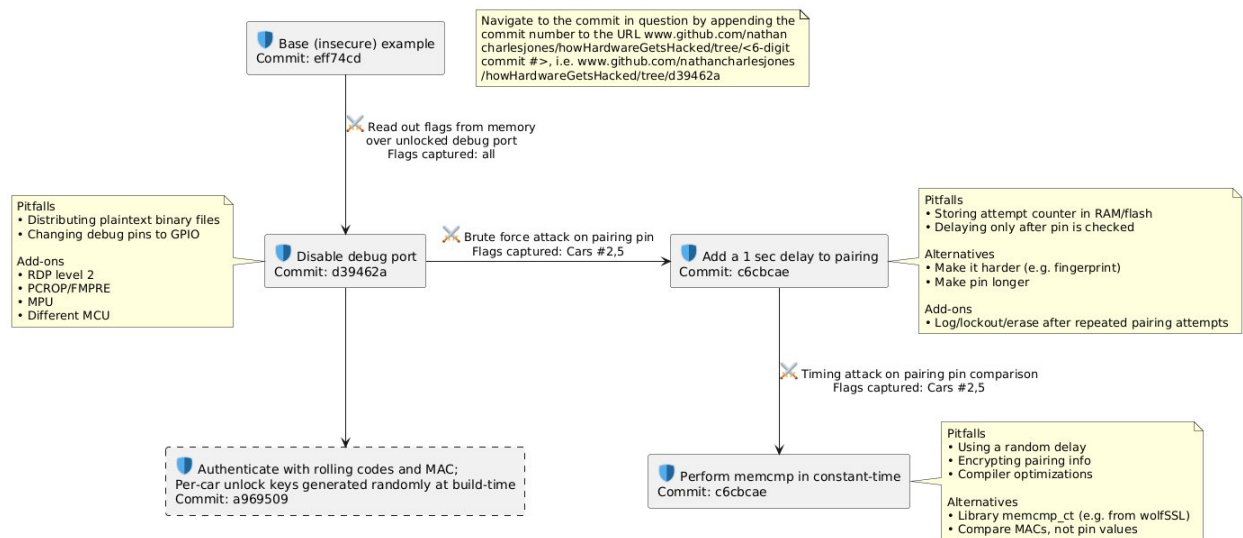
- Plaintext car binary
- Plaintext paired fob binary ←
- Protected car binary
- Protected paired fob binary
- Protected unpaired fob binary
- Pairing PIN

The attacker has full access to this car, including the unprotected binaries for the car and the paired fob. This car will not have any flags for the attacker to capture, as it represents a car that was purchased from the manufacturer.

If that paired fob has the secret key to conduct pairing operations, then an attacker can pretty easily find it in the binary; we even discussed briefly how to do something like that in [Part 4](#) of this series.

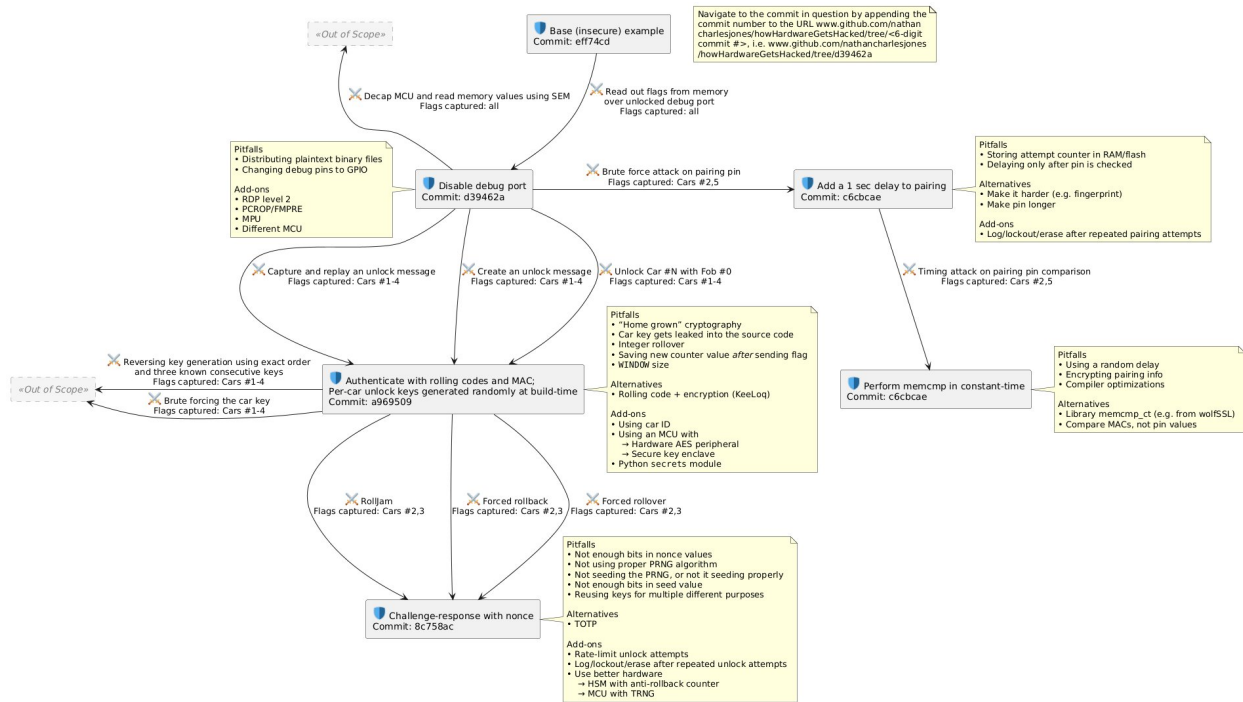
Updated threat model

We can now update our threat model with these two new attacks. The diagram is getting rather large, so I'll only show the relevant portions below. The full threat model can be seen at the end of this section.



One change from previous threat models is that I've separated "Alternatives" into "Alternatives" and "Add-ons". The former describes actual alternative solutions that would work to defend against the attack at hand, they just weren't implemented; the latter describes techniques that could augment one of the actual defenses (the one we implemented or one from the "Alternatives" list) but that don't provide a defense on their own.

Full threat model



Conclusion

/*****[🐱 Explain this article to your grandma]*****

Take a look over the article and any notes you took and then pretend your dear, sweet grandma just called and asked, “What are you reading?” and then you said, “Oh, it’s just some nerd stuff” and she said, “Of course, but I’m still interested and I’ll do my best to follow along; please, tell me what it is you’ve been reading!” and you said, “Well, okay Grandma...” How would you explain this article in terms she could understand?

*****/

The unlock processing being as secure as it is, we looked to easier paths an attacker might take and found a sorely insecure pairing process: an attacker could easily **brute force** every possible pin combination and find the right one in a matter of hours.

The straightforward defense only required adding a small delay (meaningless to a user but crippling to an attacker). However, it turned out that our MCU was leaking information the entire time by virtue of doing a byte-by-byte comparison of the pin values! This allowed an attacker to conduct a **timing attack** in which the length of time it took our MCU to send back an error message informed the attacker which byte was the first wrong byte in a guessed pin. The best defense is to use a constant-time version of memcmp, although

another solution would have been to have computed the MAC value of a received pin and compared that to the MAC value of the stored pin.

A timing attack is an example of a **side-channel attack** and there are many side-channels on an MCU, including power draw and the nature of any error messages. Our firmware doesn't seem to be susceptible to any other timing attacks right now, but this isn't the last we've seen of side-channel attacks in this article series.

If you've made it this far, thanks for reading and happy hacking!

ⁱ 100 years seems like a pretty long time, to me. At one guess per 2.5 ms, an attacker could make 1.26×10^{12} guesses in that time. To ensure that the attacker has a less-than-50% chance of guessing the correct pin in that time, we need twice as many possible values or 2.52×10^{12} values. This would therefore necessitate a pin that had 41.2 or 42 bits.